Тема 11

• Полиморфизъм – 1 име (на функция), много различни имплементации

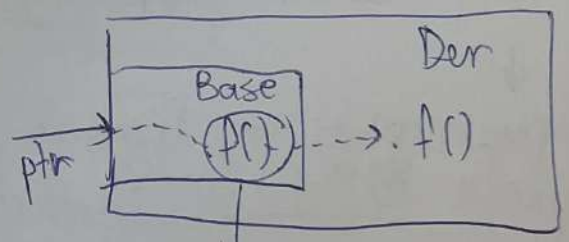
→ compile-time polymorphism – изборът на функция става по време на компилация и се определя от типа на указателя/референцията, извикал/а функцията (function/operator overloading)

→ run-time polymorphism – изборът на функция става по време на изпълнение

→ чрез виртуални функции

```
class Base {
virtual f() {cout<<"Base";}
};
class Der : public Base {
f() {cout << "Der";}
};
```

override




```
Der d;
Base* ptr = &d;
d.f(); //Der::f()
ptr->f(); //Der::f()
```

f() е виртуална ⇒ се търси най-конкретната функция, която може да се изпълни

→ презаписване виртуални функции с override

```
class A { void f() {}};
class B { void f() override {}};
```

override → не позволява компилация, ако е използвана грешно

A::f() НЕ е виртуална → комп. грешка

```
class A {
  virtual f() const {}
};
```

```
class B: public A {
  f() override { }
};
```

A::f() е const, B::f() не е → комп. грешка

- чисто-виртуална функция (pure-virtual function) – виртуална функция, която няма имплементация ⇒ е предназначена да бъде презаписана от наследниците

- абстрактен клас – клас, в който има поне една чисто-виртуална функция; не може да се създават обекти на абстрактен клас

```
class Base {
  virtual void f() = 0;
  virtual ~Base() = default;
}
```

– Base е абстрактен клас
– всеки наследник на Base, който не презапише f(), също е абстрактен

↓

Деструкторът на базовия клас в полиморфна йерархия ВИНАГИ е виртуален (иначе memory leak)

- ключова дума final

→ за функции – не може да се презаписва надолу в йерархията

→ за класове – не може да бъде наследяван

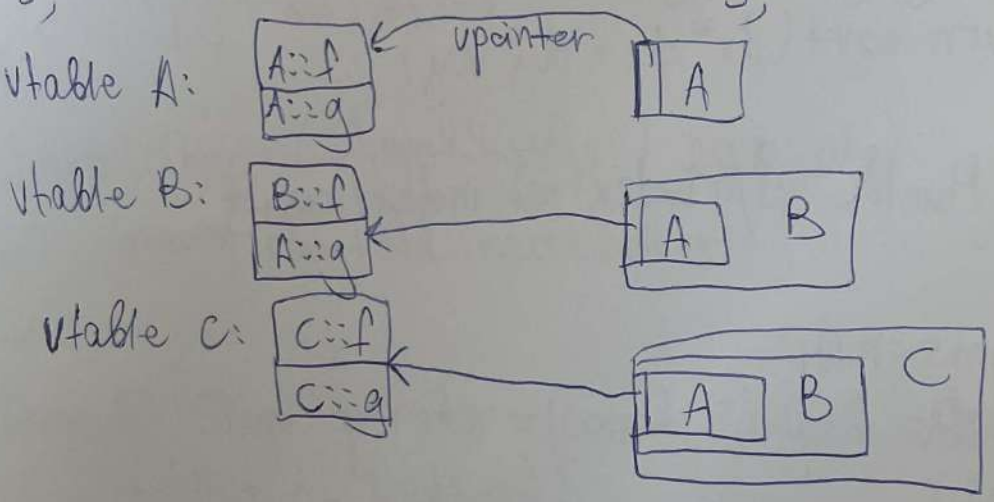
ртуални таблици – "масив от указатели към функции"

→ има vpointer (добавя се към размера на класа)

→ всеки клас, в който има виртуални функции, има своя виртуална таблица, в която се указва коя функция трябва да бъде извикана

→ всеки обект има <u>една</u> виртуална таблица, като се закача за базовия клас и стои в началото на обекта

```
class A {          class B : A {          class C : B {
  virtual f()        f() override           f() override
  virtual g()      };                       g() override
};                                        }
```



→ във виртуалните таблици има <u>само</u> виртуални ф-ии

```
A(..) : Base(...) {...}
```

↓

```
Base() {...}
```

↓

закачаме vtable A в края на Base()

```
~A() {...}
```

↓

vpointer в Base вече <u>НЕ</u> е този на A